

CODE MENTOR

Complete Development Blueprint

Feb 27, 2026 → July 2026

By 5c077

1. RECOMMENDED TECH STACK

1.1 Frontend — React Native + React (Web)

Use React Native with Expo for mobile (Android + iOS) and React.js for the web version. This gives you one JavaScript codebase for all platforms, which is ideal for a solo student developer.

- **Mobile (Android & iOS):** React Native + Expo
- **Web:** React.js + Vite
- **Styling & Animations:** Tailwind CSS + Framer Motion
- **Navigation:** React Navigation
- **Game-like Animations:** Lottie React Native

1.2 Backend — Node.js + Express

Node.js with Express is the best choice for a student: fast to learn, huge community, and JavaScript everywhere (same language as your frontend).

- **Server:** Node.js + Express.js
- **Authentication:** JWT (JSON Web Tokens)
- **Password Hashing:** bcrypt
- **Real-time features:** Socket.IO (optional)

1.3 Database — PostgreSQL (NOT SQLite)

SQLite is great for tiny local apps but NOT recommended for CODE MENTOR. Here's why PostgreSQL is better:

- **Scalability:** PostgreSQL handles many users simultaneously (SQLite locks on write)
- **Relations:** Better for user accounts, XP systems, progress tracking
- **Hosting:** Free tier available on Supabase or Railway
- **SQLite Problem:** SQLite is single-file and breaks in production

Use Supabase (free tier) which gives you PostgreSQL + Auth + Storage all in one. Or use Railway.app for PostgreSQL hosting.

1.4 AI Integration — Anthropic Claude API

For the built-in AI help assistant, use the Anthropic Claude API (claude-haiku-3-5 for cost efficiency). This powers the in-app AI mentor that answers student questions.

- **Model:** claude-haiku-3-5-20251001 for fast, cheap responses
- **Streaming:** Streaming responses for real-time typing effect
- **Context:** Context-aware prompts per programming language

1.5 VSCode Extensions to Install Now

- ES7+ React/Redux/React-Native snippets
- Prettier - Code Formatter
- ESLint
- Thunder Client (API testing)
- GitLens
- Tailwind CSS IntelliSense
- SQLTools + SQLTools PostgreSQL driver

2. INITIAL SETUP — Step by Step

2.1 Install Required Software

Install in this exact order:

```
# 1. Node.js LTS - https://nodejs.org
# 2. Install global tools
npm install -g expo-cli
npm install -g create-react-app
# 3. Install Git - https://git-scm.com
# 4. Create a free account on: supabase.com
```

2.2 Project Structure

```
code-mentor/
├─ mobile/      (React Native + Expo)
├─ web/         (React.js + Vite)
├─ backend/     (Node.js + Express)
├─ shared/      (shared types, utils)
└─ reminder/    (your daily coding reminder)
```

2.3 Bootstrap the Projects

```
# Backend
mkdir code-mentor && cd code-mentor
mkdir backend && cd backend
npm init -y
npm install express cors dotenv bcryptjs jsonwebtoken pg
npm install --save-dev nodemon

# Web Frontend
cd ..
npm create vite@latest web -- --template react
cd web && npm install
npm install axios react-router-dom framer-motion
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

# Mobile
```

```
cd ..  
npx create-expo-app mobile --template blank  
cd mobile  
npm install @react-navigation/native @react-navigation/stack  
npx expo install react-native-screens react-native-safe-area-context
```

3. DATABASE SCHEMA (PostgreSQL)

Create these tables in Supabase SQL editor or via migration files:

```
-- Users table
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  xp INTEGER DEFAULT 0,
  level INTEGER DEFAULT 1,
  avatar TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Languages & Tracks
CREATE TABLE tracks (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL, -- Python, C++, JS, Swift, AI/ML
  icon TEXT,
  description TEXT
);

-- Modules per track
CREATE TABLE modules (
  id SERIAL PRIMARY KEY,
  track_id INTEGER REFERENCES tracks(id),
  title VARCHAR(200) NOT NULL,
  order_index INTEGER NOT NULL,
  xp_required INTEGER DEFAULT 0,
  xp_reward INTEGER DEFAULT 100
);

-- Quizzes per module
CREATE TABLE quizzes (
  id SERIAL PRIMARY KEY,
  module_id INTEGER REFERENCES modules(id),
  question TEXT NOT NULL,
```

```
options JSONB NOT NULL, -- ['option1','option2',...]
correct_index INTEGER NOT NULL
);

-- User progress
CREATE TABLE user_progress (
  id SERIAL PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  module_id INTEGER REFERENCES modules(id),
  completed BOOLEAN DEFAULT FALSE,
  quiz_passed BOOLEAN DEFAULT FALSE,
  xp_earned INTEGER DEFAULT 0,
  completed_at TIMESTAMP
);
```

4. BACKEND CODE — Node.js + Express

4.1 File: backend/server.js

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const app = express();
app.use(cors({ origin: process.env.FRONTEND_URL }));
app.use(express.json());

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/tracks', require('./routes/tracks'));
app.use('/api/progress', require('./routes/progress'));
app.use('/api/ai', require('./routes/ai'));

app.listen(3001, () => console.log('Server running on 3001'));
```

4.2 File: backend/routes/auth.js

```
const router = require('express').Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { pool } = require('../db');

// REGISTER
router.post('/register', async (req, res) => {
  const { username, email, password } = req.body;
  try {
    const hash = await bcrypt.hash(password, 12);
    const result = await pool.query(
      'INSERT INTO users (username,email,password_hash) VALUES ($1,$2,$3) RETURNING id,username,email,xp,level',
      [username, email, hash]
    );
    const token = jwt.sign({ userId: result.rows[0].id }, process.env.JWT_SECRET, {
      expiresIn: '7d'
    });
    res.json({ token, user: result.rows[0] });
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'Internal server error' });
  }
});
```



```

    } catch(e) {
      if(e.code === '23505') return res.status(400).json({ error: 'Email or username
taken' });
      res.status(500).json({ error: 'Server error' });
    }
  });

// LOGIN
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  try {
    const result = await pool.query('SELECT * FROM users WHERE email=$1', [email]);
    if(!result.rows[0]) return res.status(400).json({ error: 'Invalid credentials'
});
    const valid = await bcrypt.compare(password, result.rows[0].password_hash);
    if(!valid) return res.status(400).json({ error: 'Invalid credentials' });
    const token = jwt.sign({ userId: result.rows[0].id }, process.env.JWT_SECRET, {
expiresIn: '7d' });
    const { password_hash, ...user } = result.rows[0];
    res.json({ token, user });
  } catch(e) { res.status(500).json({ error: 'Server error' }); }
});

module.exports = router;

```

4.3 File: backend/db.js

```

const { Pool } = require('pg');

const pool = new Pool({ connectionString: process.env.DATABASE_URL, ssl: {
rejectUnauthorized: false } });

module.exports = { pool };

```

4.4 File: backend/.env (NEVER commit this!)

```

DATABASE_URL=postgresql://user:password@host:5432/dbname
JWT_SECRET=your_super_secret_key_minimum_32_chars
ANTHROPIC_API_KEY=sk-ant-...
FRONTEND_URL=http://localhost:5173

```

4.5 File: backend/routes/ai.js — AI Helper Route

```

const router = require('express').Router();

```

```
const Anthropic = require('@anthropic-ai/sdk');
const { authenticate } = require('../middleware/auth');

const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

router.post('/ask', authenticate, async (req, res) => {
  const { question, language, context } = req.body;
  try {
    const message = await client.messages.create({
      model: 'claude-haiku-4-5-20251001',
      max_tokens: 1024,
      system: `You are CODE MENTOR's AI tutor. Help learners understand ${language}
programming. Be encouraging, clear, and use simple examples. Keep responses under
200 words.`,
      messages: [{ role: 'user', content: `${context ? 'Context: ' + context +
'\n\n' : ''}Question: ${question}` } ]
    });
    res.json({ answer: message.content[0].text });
  } catch(e) { res.status(500).json({ error: 'AI unavailable' }); }
});

module.exports = router;
```

5. FRONTEND CODE — React Web

5.1 File: web/src/App.jsx — Main Router

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth';
import Login from '../pages/Login';
import Register from '../pages/Register';
import Dashboard from '../pages/Dashboard';
import TrackPage from '../pages/TrackPage';
import ModulePage from '../pages/ModulePage';
import Quiz from '../pages/Quiz';

export default function App() {
  const { user, loading } = useAuth();

  if(loading) return <div className='loading-screen'><span>CODE
MENTOR</span></div>;

  return (
    <BrowserRouter>
      <Routes>
        <Route path='/' element={user ? <Navigate to='/dashboard' /> : <Navigate
to='/login' />} />
        <Route path='/login' element={<Login />} />
        <Route path='/register' element={<Register />} />
        <Route path='/dashboard' element={user ? <Dashboard /> : <Navigate
to='/login' />} />
        <Route path='/track/:trackId' element={user ? <TrackPage /> : <Navigate
to='/login' />} />
        <Route path='/module/:moduleId' element={user ? <ModulePage /> : <Navigate
to='/login' />} />
        <Route path='/quiz/:moduleId' element={user ? <Quiz /> : <Navigate
to='/login' />} />
      </Routes>
    </BrowserRouter>
  );
}
```

5.2 File: web/src/pages/Login.jsx — Login with 'By 5c077'

```
import { useState } from 'react';
import { motion } from 'framer-motion';
```

```

import { useNavigate } from 'react-router-dom';
import api from '../api';

export default function Login() {
  const [form, setForm] = useState({ email: '', password: '' });
  const [error, setError] = useState('');
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault(); setError('');
    try {
      const { data } = await api.post('/auth/login', form);
      localStorage.setItem('token', data.token);
      navigate('/dashboard');
    } catch(err) { setError(err.response?.data?.error || 'Login failed'); }
  };

  return (
    <div className='min-h-screen bg-gradient-to-br from-indigo-900 via-purple-900 to-blue-900 flex items-center justify-center'>
      <motion.div initial={{opacity:0,y:40}} animate={{opacity:1,y:0}}
        className='bg-white/10 backdrop-blur-md rounded-2xl p-10 w-full max-w-md border border-white/20'>
        <h1 className='text-4xl font-black text-white text-center mb-2'>CODE MENTOR</h1>
        <p className='text-blue-200 text-center mb-8'>Level up your coding skills</p>
        {error && <p className='text-red-400 text-sm mb-4'>{error}</p>}
        <form onSubmit={handleSubmit} className='space-y-4'>
          <input type='email' placeholder='Email' value={form.email}
            onChange={e => setForm({...form, email: e.target.value})}
            className='w-full bg-white/20 rounded-xl px-4 py-3 text-white placeholder-blue-300 outline-none border border-white/30 focus:border-blue-400' />
          <input type='password' placeholder='Password' value={form.password}
            onChange={e => setForm({...form, password: e.target.value})}
            className='w-full bg-white/20 rounded-xl px-4 py-3 text-white placeholder-blue-300 outline-none border border-white/30 focus:border-blue-400' />
          <motion.button whileHover={{scale:1.02}} whileTap={{scale:0.98}}
            type='submit'
            className='w-full bg-blue-500 hover:bg-blue-400 text-white font-bold py-3 rounded-xl transition'>
            Login
          </motion.button>
        </div>
      </div>

```

```
        </form>

        <p className='text-blue-200 text-center mt-4 text-sm'>
          No account? <a href='/register' className='text-white
underline'>Register</a>
        </p>

        {/* Required branding */}

        <p className='absolute bottom-4 right-4 text-white/30 text-xs'>By 5c077</p>
      </motion.div>
    </div>
  );
}
```

6. MOBILE APP — React Native + Expo

6.1 Running the Mobile App

```
cd mobile
npx expo start
# Scan QR code with Expo Go app on your phone
# For Android emulator: press 'a'
# For iOS simulator (Mac only): press 'i'
```

6.2 File: mobile/app/(tabs)/login.tsx — Mobile Login

The mobile login screen follows the same design as web but uses React Native components. Key differences:

- Use `<View>` instead of `<div>`
- Use `<TextInput>` instead of `<input>`
- Use `<TouchableOpacity>` instead of `<button>`
- Use `StyleSheet.create()` for styles instead of Tailwind

The login screen must also show 'By 5c077' in the bottom right corner.

6.3 Building for Production

```
# Android APK (free)
npx eas build -p android --profile preview

# iOS (requires Apple Developer Account - $99/year)
npx eas build -p ios

# Install EAS CLI first:
npm install -g eas-cli
eas login
```

7. SECURITY IMPLEMENTATION

7.1 Authentication Security

- bcrypt with 12 rounds for password hashing (not 10, not 8 — use 12)
- JWT tokens with 7-day expiry stored in httpOnly cookies for web, SecureStore for mobile
- Refresh token system: short-lived access tokens (15min) + long-lived refresh tokens (30 days)
- Rate limiting on login: max 5 attempts per IP per 15 minutes

7.2 File: backend/middleware/auth.js

```
const jwt = require('jsonwebtoken');

const authenticate = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if(!token) return res.status(401).json({ error: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch { res.status(401).json({ error: 'Invalid token' }); }
};

module.exports = { authenticate };
```

7.3 File: backend/middleware/rateLimit.js

```
npm install express-rate-limit

const rateLimit = require('express-rate-limit');
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5,
  message: { error: 'Too many login attempts. Try again in 15 minutes.' }
});

module.exports = { loginLimiter };
```

7.4 Input Validation

```
npm install express-validator
```

```
const { body, validationResult } = require('express-validator');

const registerValidation = [
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 8 }).matches(/[A-Z]/).matches(/[0-9]/),
  body('username').isLength({ min: 3, max: 20 }).isAlphanumeric(),
];
```


8. XP & GAMIFICATION SYSTEM

The XP system is what makes CODE MENTOR feel like a game. Here's how to implement it:

8.1 XP Rewards Table

Action	XP Reward	Notes
Complete a module lesson	+50 XP	Per lesson
Pass a quiz (first try)	+100 XP	Bonus for first try
Pass a quiz (retry)	+50 XP	Reduced reward
Daily streak (login)	+20 XP	Per day
Complete a full track	+500 XP	Track completion
Help another user (future)	+30 XP	Community feature

8.2 Level Thresholds

```
// backend/utils/xp.js
const LEVELS = [
  0,      // Level 1
  200,    // Level 2
  500,    // Level 3
  1000,   // Level 4
  2000,   // Level 5 - 'Code Apprentice'
  4000,   // Level 6
  7000,   // Level 7 - 'Code Wizard'
  12000,  // Level 8
  20000,  // Level 9
  35000,  // Level 10 - 'CODE MASTER'
];

const getLevelFromXP = (xp) => {
  let level = 1;
  for(let i = 0; i < LEVELS.length; i++) {
    if(xp >= LEVELS[i]) level = i + 1;
  }
  return level;
};
```


9. DEVELOPMENT ROADMAP (Feb 27 → July 2026)

Based on your schedule: full days on Sundays, 2-3 hours on other days. This is realistic and achievable!

Phase 1: Foundation (Week 1-4, Feb 27 - Mar 27)

Week	Dates	Tasks & Deliverables
1	Feb 27 - Mar 1	Setup environment: install Node, Expo, VSCode extensions. Create project folders. Create Supabase account and database.
2	Mar 2 - Mar 8	Build backend: server.js, database connection, auth routes (register/login). Test with Thunder Client.
3	Mar 9 - Mar 15	Build web Login & Register pages with animations. Connect to backend. Store JWT token.
4	Mar 16 - Mar 22	Build Dashboard page showing tracks (Python, JS, C++, Swift, AI/ML). XP bar component.
5	Mar 23 - Mar 27	Build Track page showing modules list with locked/unlocked states based on XP.

Phase 2: Core Learning (Week 5-9, Mar 28 - May 1)

Week	Dates	Tasks & Deliverables
1	Mar 28 - Apr 3	Build Module lesson page with code examples, syntax highlighting (prism.js). Build lesson content for Python basics.
2	Apr 4 - Apr 10	Build Quiz component with animated questions, answer feedback, XP award on pass.
3	Apr 11 - Apr 17	Implement XP system backend: award XP, check level up, update user progress.
4	Apr 18 - Apr 24	Build AI Helper chat panel using Anthropic API. Implement streaming response display.
5	Apr 25 - May 1	Add content: Python track modules 1-5 with quizzes. Build progress tracking.

Phase 3: All Languages (Week 10-14, May 2 - Jun 5)

Week	Dates	Tasks & Deliverables
1	May 2 - May 8	JavaScript track: 5 modules + quizzes. Add code editor (Monaco Editor) for interactive coding.

2	May 9 - May 15	C++ track: 5 modules + quizzes. Achievement badges system.
3	May 16 - May 22	Swift track for iOS: 5 modules + quizzes. Leaderboard page.
4	May 23 - May 29	AI/ML track: Python ML, model training basics, 5 modules + quizzes.
5	May 30 - Jun 5	Daily streak system, notifications, profile page with stats and badges.

Phase 4: Mobile + Polish (Week 15-19, Jun 6 - July 10)

Week	Dates	Tasks & Deliverables
1	Jun 6 - Jun 12	Build React Native mobile version. Login/Register screens.
2	Jun 13 - Jun 19	Mobile Dashboard and Track pages. Navigation. XP display.
3	Jun 20 - Jun 26	Mobile Module and Quiz pages. AI helper in mobile.
4	Jun 27 - Jul 3	Performance optimization, error handling, loading states everywhere.
5	Jul 4 - Jul 10	Build Android APK with EAS. Test on real device. Fix bugs.

10. DAILY CODING REMINDER APP

Save this as reminder/reminder.py and run it in the background on your PC every day.

```
# reminder/reminder.py
import tkinter as tk
from tkinter import messagebox
import datetime
import threading
import time

def get_message():
    day = datetime.datetime.now().weekday() # 6 = Sunday
    hour = datetime.datetime.now().hour
    days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
    if day == 6: # Sunday
        return f'🦉 It\'s SUNDAY - Full coding day for CODE MENTOR!\nBlock your whole day. No excuses today! 🧐'
    else:
        if hour < 12:
            return f'📅 Good morning! Remember: 2-3 hours on CODE MENTOR today ({days[day]})\nEvery session counts!'
        elif hour < 18:
            return f'🕒 Afternoon reminder! Have you coded today ({days[day]})?\nCODE MENTOR needs you - 2-3 hours!'
        else:
            return f'🌙 Evening! Last chance to code today ({days[day]})\nEven 1 hour of CODE MENTOR counts!'

def show_reminder():
    root = tk.Tk()
    root.withdraw()
    msg = get_message()
    messagebox.showinfo('CODE MENTOR Reminder', msg)
    root.destroy()

def reminder_loop():
    while True:
        now = datetime.datetime.now()
```

```
# Remind at 9:00, 14:00, and 20:00
if now.minute == 0 and now.hour in [9, 14, 20]:
    show_reminder()
    time.sleep(61) # Avoid double trigger
time.sleep(30)

if __name__ == '__main__':
    print('CODE MENTOR Reminder running...')
    print('You will be reminded at 9:00, 14:00, and 20:00 every day.')
    reminder_loop()
```

10.1 Run on Startup (Windows)

```
# Create a .bat file: reminder/start_reminder.bat
@echo off
pythonw reminder.py

# Add to startup: Win+R → shell:startup → paste the .bat file there
```

10.2 Run on Startup (Mac)

```
# Create a launchd plist or simply add to Login Items:
# System Preferences → Users & Groups → Login Items → Add reminder.py
```

10.3 Build to .exe (Windows, optional)

```
pip install pyinstaller
pyinstaller --onefile --noconsole reminder.py
# Output: dist/reminder.exe
```

11. FINAL CHECKLIST

Backend Checklist

- Express server running on port 3001
- PostgreSQL connected via Supabase
- Auth routes: /api/auth/register and /api/auth/login
- JWT middleware protecting private routes
- Rate limiting on login endpoint
- Input validation with express-validator
- AI helper route connected to Anthropic API
- XP award and progress tracking routes

Frontend Checklist

- Login page with 'By 5c077' in bottom right corner
- Animated transitions between pages (Framer Motion)
- XP bar and level display on dashboard
- Track selection page with all 5 languages
- Module page with code syntax highlighting
- Quiz component with animated feedback
- AI helper chat panel
- Profile page with achievements and badges

Mobile Checklist

- Expo project with React Navigation
- All key screens ported from web
- Tested on real Android device via Expo Go
- Android APK built with EAS

Security Checklist

- Passwords hashed with bcrypt (rounds: 12)
- JWT tokens with expiry
- HTTPS in production (Render/Railway handle this)
- Rate limiting active on auth routes
- Input sanitized on all endpoints
- .env file added to .gitignore

Good luck, 5c077! You've got this 🚀

CODE MENTOR — By 5c077